第7章 复 用 类

复用代码是Java众多引人注目的功能之一。但要想成为极具革命性的语言，仅仅能够复制代码并对之加以改变是不够的，它还必须能够做更多的事情。

上述方法常为C这类过程型语言所使用，但收效并不是很好。正如Java中所有事物一样，问题解决都是围绕着类展开的。可以通过创建新类来复用代码，而不必再重头开始编写。可以使用别人业已开发并调试好的类。

此方法的窍门在于使用类而不破坏现有程序代码。读者将会在本章中看到两种达到这一目的的方法。第一种方法非常直观：只需在新的类中产生现有类的对象。由于新的类是由现有类的对象所组成，所以这种方法称为组合。该方法只是复用了现有程序代码的功能，而非它的形式。

第二种方法则更细致一些，它按照现有类的类型来创建新类。无需改变现有类的形式，采用现有类的形式并在其中添加新代码。这种神奇的方式称为继承，而且编译器可以完成其中大部分工作。继承是面向对象程序设计的基石之一，我们将在第8章中探究其含义与功能。

就组合和继承而言，其语法和行为大多是相似的。由于它们是利用现有类型生成新类型，所以这样做极富意义。在本章中，读者将会了解到这两种代码重用机制。

7.1 组合语法

本书到目前为止，已多次使用组合技术。只需将对象引用置于新类中即可。例如，假设你需要某个对象，它要具有多个**String**对象、几个基本类型数据，以及另一个类的对象。对于非基本类型的对象，必须将其引用置于新的类中，但可以直接定义基本类型数据：


```
//: reusing/SprinklerSystem.java
// Composition for code reuse.

class WaterSource {
  private String s;
  WaterSource() {
    System.out.println("WaterSource()");
    s = "Constructed";
  }
  public String toString() { return s; }
}

public class SprinklerSystem {
  private String valve1, valve2, valve3, valve4;
  private WaterSource source = new WaterSource();
  private int i;
  private float f;
  public String toString() {
    return
      "valve1 = " + valve1 + " " +
      "valve2 = " + valve2 + " " +
      "valve3 = " + valve3 + " " +
      "valve4 = " + valve4 + "\n" +
      "i = " + i + " " + "f = " + f + " " +
      "source = " + source;
  }
```

```
  public static void main(String[] args) {
    SprinklerSystem sprinklers = new SprinklerSystem();
    System.out.println(sprinklers);
  }
} /* Output:
WaterSource()
valve1 = null valve2 = null valve3 = null valve4 = null
i = 0 f = 0.0 source = Constructed
*///:~
```

在上面两个类所定义的方法中，有一个很特殊：**toString()**。每一个非基本类型的对象都有一个**toString()**方法，而且当编译器需要一个**String**而你却只有一个对象时，该方法便会被调用。所以在**SprinklerSystem.toString()**的表达式中：

```
"source = " + source;
```

编译器将会得知你想要将一个**String**对象（**"source="**）同**WaterSource**相加。由于只能将一个String对象和另一个**String**对象相加，因此编译器会告诉你："我将调用**toString()**，把**source**转换成为一个**String**！"这样做之后，它就能够将两个**String**连接到一起并将结果传递给**System.out.println()**（或者使用与此等价的本书中静态的**print()**和**printnb()**方法）。每当想要使所创建的类具备这样的行为时，仅需要编写一个**toString()**方法即可。

238

正如我们在第2章中所提到的，类中域为基本类型时能够自动被初始化为零。但是对象引用会被初始化为**null**，而且如果你试图为它们调用任何方法，都会得到一个异常——运行时错误。很方便的是，在不抛出异常的情况下仍旧可以打印一个**null**引用。

编译器并不是简单地为每一个引用都创建默认对象，这一点是很有意义的，因为若真要那样做的话，就会在许多情况下增加不必要的负担。如果想初始化这些引用，可以在代码中的下列位置进行：

1. 在定义对象的地方。这意味着它们总是能够在构造器被调用之前被初始化。

2. 在类的构造器中。

3. 就在正要使用这些对象之前，这种方式称为*惰性初始化*。在生成对象不值得及不必每次都生成对象的情况下，这种方式可以减少额外的负担。

4. 使用实例初始化。

以下是这四种方式的示例：

```
//: reusing/Bath.java
// Constructor initialization with composition.
import static net.mindview.util.Print.*;

class Soap {
  private String s;
  Soap() {
    print("Soap()");
    s = "Constructed";
  }
  public String toString() { return s; }
}

public class Bath {
  private String // Initializing at point of definition:
    s1 = "Happy",
    s2 = "Happy",
    s3, s4;
  private Soap castille;
  private int i;
  private float toy;
  public Bath() {
```

239

```
    print("Inside Bath()");
    s3 = "Joy";
    toy = 3.14f;
    castille = new Soap();
  }
  // Instance initialization:
  { i = 47; }
  public String toString() {
    if(s4 == null) // Delayed initialization:
      s4 = "Joy";
    return
      "s1 = " + s1 + "\n" +
      "s2 = " + s2 + "\n" +
      "s3 = " + s3 + "\n" +
      "s4 = " + s4 + "\n" +
      "i = " + i + "\n" +
      "toy = " + toy + "\n" +
      "castille = " + castille;
  }
  public static void main(String[] args) {
    Bath b = new Bath();
    print(b);
  }
} /* Output:
Inside Bath()
Soap()
s1 = Happy
s2 = Happy
s3 = Joy
s4 = Joy
i = 47
toy = 3.14
castille = Constructed
*///:~
```

请注意，在**Bath**的构造器中，有一行语句在所有初始化产生之前就已经执行了。如果没有在定义处初始化，那么除非发生了不可避免的运行期异常，否则将不能保证信息在发送给对象引用之前已经被初始化。 240

当**toString()**被调用时，它将填充**s4**的值，以确保所有的域在使用之时已被妥善初始化。

练习1：(2) 创建一个简单的类。在第二个类中，将一个引用定义为第一个类的对象。运用惰性初始化来实例化这个对象。

7.2 继承语法

继承是所有OOP语言和Java语言不可缺少的组成部分。当创建一个类时，总是在继承，因此，除非已明确指出要从其他类中继承，否则就是在隐式地从Java的标准根类**Object**进行继承。

组合的语法比较平实，但是继承使用的是一种特殊的语法。在继承过程中，需要先声明“新类与旧类相似”。这种声明是通过在类主体的左边花括号之前，书写后面紧随基类名称的关键字**extends**而实现的。当这么做时，会自动得到基类中所有的域和方法。例如：

```
//: reusing/Detergent.java
// Inheritance syntax & properties.
import static net.mindview.util.Print.*;

class Cleanser {
  private String s = "Cleanser";
  public void append(String a) { s += a; }
  public void dilute() { append(" dilute()"); }
  public void apply() { append(" apply()"); }
  public void scrub() { append(" scrub()"); }
  public String toString() { return s; }
```

```
    public static void main(String[] args) {
      Cleanser x = new Cleanser();
      x.dilute(); x.apply(); x.scrub();
      print(x);
    }
  }

  public class Detergent extends Cleanser {
    // Change a method:
    public void scrub() {
      append(" Detergent.scrub()");
      super.scrub(); // Call base-class version
    }
    // Add methods to the interface:
    public void foam() { append(" foam()"); }
    // Test the new class:
    public static void main(String[] args) {
      Detergent x = new Detergent();
      x.dilute();
      x.apply();
      x.scrub();
      x.foam();
      print(x);
      print("Testing base class:");
      Cleanser.main(args);
    }
  } /* Output:
  Cleanser dilute() apply() Detergent.scrub() scrub() foam()
  Testing base class:
  Cleanser dilute() apply() scrub()
  *///:~
```

241

这个程序示范了Java 的许多特性。首先，在**Cleanser** 的**append()**方法中，我们用“+=”操作符将几个**String**对象连接成s，此操作符是被Java设计者重载用以处理**String**对象的操作符之一（另一个是“+”）。

其次，**Cleanser**和**Detergent**均含有**main()**方法。可以为每个类都创建一个**main()**方法。这种在每个类中都设置一个**main()**方法的技术可使每个类的单元测试都变得简便易行。而且在完成单元测试之后，也无需删除**main()**，可以将其留待下次测试。

即使是一个程序中含有多个类，也只有命令行所调用的那个类的**main()**方法会被调用。因此，在此例中，如果命令行是**java Detergent**，那么**Detergent.main()**将会被调用。即使**Cleanser**不是一个**public** 类，如果命令行是**java Cleanser**，那么**Cleanser.main()** 仍然会被调用。即使一个类只具有包访问权限，其**public main()**仍然是可访问的。

242

在此例中，可以看到**Detergent.main()**明确调用了**Cleanser.main()**，并将从命令行获取的参数传递给了它。当然，也可以向其传递任意的**String** 数组。

Cleanser中所有的方法都必须是**public**的，这一点非常重要。请记住，如果没有加任何访问权限修饰词，那么成员默认的访问权限是包访问权限，它仅允许包内的成员访问。因此，在此包中，如果没有访问权限修饰词，任何人都可以使用这些方法。例如，**Detergent**就不成问题。但是，其他包中的某个类若要从**Cleanser**中继承，则只能访问**public**成员。所以，为了继承，一般的规则是将所有的数据成员都指定为**private**，将所有的方法指定为**public**（稍后将会学到，**protected**成员也可以借助导出类来访问）。当然，在特殊情况下，必须做出调整，但上述方法的确是一个很有用的规则。

在**Cleanser**的接口中有一组方法：**append()**、**dilute()**、**apply()**、**scrub()**和**toString()**。由于**Detergent**是由关键字**extends**从**Cleanser**导出的，所以它可以在其接口中自动获得这些方法，尽

管并不能看到这些方法在**Detergent**中的显式定义。因此，可以将继承视作是对类的复用。

正如我们在**scrub()**中所见，使用基类中定义的方法及对它进行修改是可行的。在此例中，你可能想要在新版本中调用从基类继承而来的方法。但是在**scrub()**中，并不能直接调用**scrub()**，因为这样做将会产生递归，而这并不是你所期望的。为解决此问题，Java用**super**关键字表示超类的意思，当前类就是从超类继承来的。为此，表达式**super.scrub()**将调用基类版本的**scrub()**方法。

在继承的过程中，并不一定非得使用基类的方法。也可以在导出类中添加新方法，其添加方式与在类中添加任意方法一样，即对其加以定义即可。**foam()**方法即为一例。

读者在**Detergent.main()**中会发现，对于一个**Detegent**对象而言，除了可以调用**Detergent**的方法（即**foam()**）之外，还可以调用**Cleanser**中所有可用的方法。

练习2：(2) 从**Detergent**中继承产生一个新的类。覆盖**scrub()**并添加一个名为**sterilize()**的新方法。

243

7.2.1 初始化基类

由于现在涉及基类和导出类这两个类，而不是只有一个类，所以要试着想像导出类所产生的结果对象，会有点困惑。从外部来看，它就像是一个与基类具有相同接口的新类，或许还会有一些额外的方法和域。但继承并不只是复制基类的接口。当创建了一个导出类的对象时，该对象包含了一个基类的子对象。这个子对象与你用基类直接创建的对象是一样的。二者区别在于，后者来自于外部，而基类的子对象被包装在导出类对象内部。

当然，对基类子对象的正确初始化也是至关重要的，而且也仅有一种方法来保证这一点：在构造器中调用基类构造器来执行初始化，而基类构造器具有执行基类初始化所需要的所有知识和能力。Java会自动在导出类的构造器中插入对基类构造器的调用。下例展示了上述机制在三层继承关系上是如何工作的：

```
//: reusing/Cartoon.java
// Constructor calls during inheritance.
import static net.mindview.util.Print.*;

class Art {
  Art() { print("Art constructor"); }
}

class Drawing extends Art {
  Drawing() { print("Drawing constructor"); }
}

public class Cartoon extends Drawing {
  public Cartoon() { print("Cartoon constructor"); }
  public static void main(String[] args) {
    Cartoon x = new Cartoon();
  }
} /* Output:
Art constructor
Drawing constructor
Cartoon constructor
*///:~
```

244

读者会发现，构建过程是从基类“向外”扩散的，所以基类在导出类构造器可以访问它之前，就已经完成了初始化。即使你不为**Cartoon()**创建构造器，编译器也会为你合成一个默认的构造器，该构造器将调用基类的构造器。

练习3：(2) 证明前面这句话。

练习4：(2) 证明基类构造器：(a) 总是会被调用；(b) 在导出类构造器之前被调用。

练习5：(1) 创建两个带有默认构造器（空参数列表）的类**A**和类**B**。从**A**中继承产生一个名为**C**的新类，并在**C**内创建一个**B**类的成员。不要给**C**编写构造器。创建一个**C**类的对象并观察其结果。

带参数的构造器

上例中各个类均含有默认的构造器，即这些构造器都不带参数。编译器可以轻松地调用它们是因为不必考虑要传递什么样的参数的问题。但是，如果没有默认的基类构造器，或者想调用一个带参数的基类构造器，就必须用关键字**super**显式地编写调用基类构造器的语句，并且配以适当的参数列表：

```
//: reusing/Chess.java
// Inheritance, constructors and arguments.
import static net.mindview.util.Print.*;

class Game {
  Game(int i) {
    print("Game constructor");
  }
}

class BoardGame extends Game {
  BoardGame(int i) {
    super(i);
    print("BoardGame constructor");
  }
}

public class Chess extends BoardGame {
  Chess() {
    super(11);
    print("Chess constructor");
  }
  public static void main(String[] args) {
    Chess x = new Chess();
  }
} /* Output:
Game constructor
BoardGame constructor
Chess constructor
*///:~
```

245

如果不在**BoardGame()**中调用基类构造器，编译器将“抱怨”无法找到符合**Game()**形式的构造器。而且，调用基类构造器必须是你在导出类构造器中要做的第一件事（如果你做错了，编译器会提醒你）。

练习6：(1) 用**Chess.java**来证明前一段话。

练习7：(1) 修改练习5，使**A**和**B**以带参数的构造器取代默认的构造器。为**C**写一个构造器，并在其中执行所有的初始化。

练习8：(1) 创建一个基类，它仅有一个非默认构造器；再创建一个导出类，它带有默认构造器和非默认构造器。在导出类的构造器中调用基类的构造器。

练习9：(2) 创建一个**Root** 类，令其含有名为**Component 1**、**Component 2**、**Component 3**的类的各一个实例（这些也由你写）。从**Root**中派生一个类**Stem**，也含有上述各“组成部分”。所有的类都应带有可打印出类的相关信息的默认构造器。

练习10：(1) 修改练习10，使每个类都仅具有非默认的构造器。

7.3 代理

第三种关系称为代理，Java并没有提供对它的直接支持。这是继承与组合之间的中庸之道，

因为我们将一个成员对象置于所要构造的类中（就像组合），但与此同时我们在新类中暴露了该成员对象的所有方法（就像继承）。例如，太空船需要一个控制模块：

246

```
//: reusing/SpaceShipControls.java

public class SpaceShipControls {
  void up(int velocity) {}
  void down(int velocity) {}
  void left(int velocity) {}
  void right(int velocity) {}
  void forward(int velocity) {}
  void back(int velocity) {}
  void turboBoost() {}
} ///:~
```

构造太空船的一种方式是使用继承：

```
//: reusing/SpaceShip.java

public class SpaceShip extends SpaceShipControls {
  private String name;
  public SpaceShip(String name) { this.name = name; }
  public String toString() { return name; }
  public static void main(String[] args) {
    SpaceShip protector = new SpaceShip("NSEA Protector");
    protector.forward(100);
  }
} ///:~
```

然而，**SpaceShip**并非真正的**SpaceShipControls**类型，即便你可以“告诉”**SpaceShip**向前运动（**forward()**）。更准确地讲，**SpaceShip**包含**SpaceShipControls**，与此同时，**SpaceShipControls**的所有方法在**SpaceShip**中都暴露了出来。代理解决了此难题：

```
//: reusing/SpaceShipDelegation.java

public class SpaceShipDelegation {
  private String name;
  private SpaceShipControls controls =
    new SpaceShipControls();
  public SpaceShipDelegation(String name) {
    this.name = name;
  }
  // Delegated methods:
  public void back(int velocity) {
    controls.back(velocity);
  }
  public void down(int velocity) {
    controls.down(velocity);
  }
  public void forward(int velocity) {
    controls.forward(velocity);
  }
  public void left(int velocity) {
    controls.left(velocity);
  }
  public void right(int velocity) {
    controls.right(velocity);
  }
  public void turboBoost() {
    controls.turboBoost();
  }
  public void up(int velocity) {
    controls.up(velocity);
  }
  public static void main(String[] args) {
```

247

```
    SpaceShipDelegation protector =
      new SpaceShipDelegation("NSEA Protector");
    protector.forward(100);
  }
} ///:~
```

可以看到，上面的方法是如何转递给了底层的**controls**对象，而其接口由此也就与使用继承得到的接口相同了。但是，我们使用代理时可以拥有更多的控制力，因为我们可以选择只提供在成员对象中的方法的某个子集。

尽管Java语言不直接支持代理，但是很多开发工具却支持代理。例如，使用JetBrains Idea IDE就可以自动生成上面的例子。

248 **练习11**：(3) 修改**Detergent.java**，让它使用代理。

7.4 结合使用组合和继承

同时使用组合和继承是很常见的事。下例就展示了同时使用这两种技术，并配以必要的构造器初始化，来创建更加复杂的类：

```
//: reusing/PlaceSetting.java
// Combining composition & inheritance.
import static net.mindview.util.Print.*;

class Plate {
  Plate(int i) {
    print("Plate constructor");
  }
}

class DinnerPlate extends Plate {
  DinnerPlate(int i) {
    super(i);
    print("DinnerPlate constructor");
  }
}

class Utensil {
  Utensil(int i) {
    print("Utensil constructor");
  }
}

class Spoon extends Utensil {
  Spoon(int i) {
    super(i);
    print("Spoon constructor");
  }
}

class Fork extends Utensil {
  Fork(int i) {
    super(i);
    print("Fork constructor");
  }
}

class Knife extends Utensil {
  Knife(int i) {
    super(i);
    print("Knife constructor");
  }
}

// A cultural way of doing something:
```

249

```
class Custom {
  Custom(int i) {
    print("Custom constructor");
  }
}

public class PlaceSetting extends Custom {
  private Spoon sp;
  private Fork frk;
  private Knife kn;
  private DinnerPlate pl;
  public PlaceSetting(int i) {
    super(i + 1);
    sp = new Spoon(i + 2);
    frk = new Fork(i + 3);
    kn = new Knife(i + 4);
    pl = new DinnerPlate(i + 5);
    print("PlaceSetting constructor");
  }
  public static void main(String[] args) {
    PlaceSetting x = new PlaceSetting(9);
  }
} /* Output:
Custom constructor
Utensil constructor
Spoon constructor
Utensil constructor
Fork constructor
Utensil constructor
Knife constructor
Plate constructor
DinnerPlate constructor
PlaceSetting constructor
*///:~
```

250

虽然编译器强制你去初始化基类，并且要求你要在构造器起始处就要这么做，但是它并不监督你必须将成员对象也初始化，因此在这一点上你自己必须时刻注意。

这些类如此清晰地分离着实使人惊讶。甚至不需要这些方法的源代码就可以复用这些代码，我们至多只需要导入一个包。(对于继承与组合来说都是如此。)

7.4.1 确保正确清理

Java中没有C++中析构函数的概念。析构函数是一种在对象被销毁时可以被自动调用的函数。其原因可能是因为在Java中，我们的习惯只是忘掉而不是销毁对象，并且让垃圾回收器在必要时释放其内存。

通常这样做是好事，但有时类可能要在其生命周期内执行一些必需的清理活动。正如我们在第5章中所提到的那样，你并不知道垃圾回收器何时将会被调用，或者它是否将被调用。因此，如果你想要某个类清理一些东西，就必须显式地编写一个特殊方法来做这件事，并要确保客户端程序员知晓他们必须要调用这一方法。就像在第12章所描述的那样，其首要任务就是，必须将这一清理动作置于finally子句之中，以预防异常的出现。

请思考一下下面这个能在屏幕上绘制图案的计算机辅助设计系统示例：

```
//: reusing/CADSystem.java
// Ensuring proper cleanup.
package reusing;
import static net.mindview.util.Print.*;

class Shape {
  Shape(int i) { print("Shape constructor"); }
  void dispose() { print("Shape dispose"); }
}
```

```
class Circle extends Shape {
  Circle(int i) {
    super(i);
    print("Drawing Circle");
  }
  void dispose() {
    print("Erasing Circle");
    super.dispose();
  }
}

class Triangle extends Shape {
  Triangle(int i) {
    super(i);
    print("Drawing Triangle");
  }
  void dispose() {
    print("Erasing Triangle");
    super.dispose();
  }
}

class Line extends Shape {
  private int start, end;
  Line(int start, int end) {
    super(start);
    this.start = start;
    this.end = end;
    print("Drawing Line: " + start + ", " + end);
  }
  void dispose() {
    print("Erasing Line: " + start + ", " + end);
    super.dispose();
  }
}

public class CADSystem extends Shape {
  private Circle c;
  private Triangle t;
  private Line[] lines = new Line[3];
  public CADSystem(int i) {
    super(i + 1);
    for(int j = 0; j < lines.length; j++)
      lines[j] = new Line(j, j*j);
    c = new Circle(1);
    t = new Triangle(1);
    print("Combined constructor");
  }
  public void dispose() {
    print("CADSystem.dispose()");
    // The order of cleanup is the reverse
    // of the order of initialization:
    t.dispose();
    c.dispose();
    for(int i = lines.length - 1; i >= 0; i--)
      lines[i].dispose();
    super.dispose();
  }
  public static void main(String[] args) {
    CADSystem x = new CADSystem(47);
    try {
      // Code and exception handling...
    } finally {
      x.dispose();
    }
  }
} /* Output:
```

251

252

```
Shape constructor
Shape constructor
Drawing Line: 0, 0
Shape constructor
Drawing Line: 1, 1
Shape constructor
Drawing Line: 2, 4
Shape constructor
Drawing Circle
Shape constructor
Drawing Triangle
Combined constructor
CADSystem.dispose()
Erasing Triangle
Shape dispose
Erasing Circle
Shape dispose
Erasing Line: 2, 4
Shape dispose
Erasing Line: 1, 1
Shape dispose
Erasing Line: 0, 0
Shape dispose
Shape dispose
*///:~
```

253

此系统中的一切都是某种**Shape**(**Shape**自身就是一种**Object**,因为**Shape**继承自根类**Object**)。每个类都覆写**Shape**的**dispose()**方法，并运用**super**来调用该方法的基类版本。尽管对象生命期中任何被调用的方法都可以做一些必需的清理工作，但是**Circle**、**Triangle**和**Line**这些特定的**Shape**类仍然都带有可以进行“绘制”的构造器。每个类都有自己的**dispose()**方法将未存于内存之中的东西恢复到对象存在之前的状态。

在**main()**中可以看到**try**和**finally**这两个之前还没有看到过的关键字，我们将在第12章对它们进行详细解释。关键字**try**表示，下面的块（用一组大括号括起来的范围）是所谓的*保护区*(guarded region)，这意味着它需要被特殊处理。其中一项特殊处理就是无论**try**块是怎样退出的，保护区后的**finally**子句中的代码总是要被执行的。这里**finally**子句表示的是“无论发生什么事，一定要为**x**调用**dispose()**”。

在清理方法（**dispose()**）中，还必须注意对基类清理方法和成员对象清理方法的调用顺序，以防某个子对象依赖于另一个子对象情形的发生。一般而言，所采用的形式应该与C++编译器在其析构函数上所施加的形式相同：首先，执行类的所有特定的清理动作，其顺序同生成顺序相反（通常这就要求基类元素仍旧存活）；然后，就如我们所示范的那样，调用基类的清理方法。

许多情况下，清理并不是问题，仅需让垃圾回收器完成该动作就行。但当必须亲自处理清理时，就得多做努力并多加小心。因为，一旦涉及垃圾回收，能够信赖的事就不会很多了。垃圾回收器可能永远也无法被调用，即使被调用，它也可能以任何它想要的顺序来回收对象。最好的办法是除了内存以外，不能依赖垃圾回收器去做任何事。如果需要进行清理，最好是编写你自己的清理方法，但不要使用**finalize()**。

练习12：(3) 将一个适当的**dispose()**方法的层次结构添加到练习9的所有类中。

254

7.4.2 名称屏蔽

如果Java的基类拥有某个已被多次重载的方法名称，那么在导出类中重新定义该方法名称并不会屏蔽其在基类中的任何版本（这一点与C++不同）。因此，无论是在该层或者它的基类中对方法进行定义，重载机制都可以正常工作：

```
//: reusing/Hide.java
// Overloading a base-class method name in a derived
// class does not hide the base-class versions.
import static net.mindview.util.Print.*;

class Homer {
  char doh(char c) {
    print("doh(char)");
    return 'd';
  }
  float doh(float f) {
    print("doh(float)");
    return 1.0f;
  }
}

class Milhouse {}

class Bart extends Homer {
  void doh(Milhouse m) {
    print("doh(Milhouse)");
  }
}

public class Hide {
  public static void main(String[] args) {
    Bart b = new Bart();
    b.doh(1);
    b.doh('x');
    b.doh(1.0f);
    b.doh(new Milhouse());
  }
} /* Output:
doh(float)
doh(char)
doh(float)
doh(Milhouse)
*///:~
```

255

可以看到，虽然**Bart**引入了一个新的重载方法（在C++中若要完成这项工作则需要屏蔽基类方法），但是在**Bart**中**Homer**的所有重载方法都是可用的。正如读者将在下一章所看到的，使用与基类完全相同的特征签名及返回类型来覆盖具有相同名称的方法，是一件极其平常的事。但它也令人迷惑不解（这也就是为什么C++不允许这样做的原因所在–防止你可能会犯错误）。

Java SE5新增加了**@Override**注解，它并不是关键字，但是可以把它当作关键字使用。当你想要覆写某个方法时，可以选择添加这个注解，在你不留心重载而并非覆写了该方法时，编译器就会生成一条错误消息：

```
//: reusing/Lisa.java
// {CompileTimeError} (Won't compile)

class Lisa extends Homer {
  @Override void doh(Milhouse m) {
    System.out.println("doh(Milhouse)");
  }
} ///:~
```

{CompileTimeError}标签把该文件从本书的Ant构建中排除了出来，但是如果手工编译该文件，就会看到下面的错误消息：

```
method does not override a method from its superclass
```

这样，**@Override**注解可以防止你在不想重载时而意外地进行了重载。

练习13：(2) 创建一个类，它应带有一个被重载了三次的方法。继承产生一个新类，并添加

一个该方法的新的重载定义，展示这四个方法在导出类中都是可以使用的。

7.5 在组合与继承之间选择

组合和继承都允许在新的类中放置子对象，组合是显式地这样做，而继承则是隐式地做。读者或许想知道二者间的区别何在，以及怎样在二者之间做出选择。

256

组合技术通常用于想在新类中使用现有类的功能而非它的接口这种情形。即，在新类中嵌入某个对象，让其实现所需要的功能，但新类的用户看到的只是为新类所定义的接口，而非所嵌入对象的接口。为取得此效果，需要在新类中嵌入一个现有类的**private**对象。

有时，允许类的用户直接访问新类中的组合成分是极具意义的；也就是说，将成员对象声明为**public**。如果成员对象自身都隐藏了具体实现，那么这种做法是安全的。当用户能够了解到你正在组装一组部件时，会使得端口更加易于理解。**car**对象即为一个很好的例子：

```
//: reusing/Car.java
// Composition with public objects.

class Engine {
  public void start() {}
  public void rev() {}
  public void stop() {}
}

class Wheel {
  public void inflate(int psi) {}
}

class Window {
  public void rollup() {}
  public void rolldown() {}
}

class Door {
  public Window window = new Window();
  public void open() {}
  public void close() {}
}

public class Car {
  public Engine engine = new Engine();
  public Wheel[] wheel = new Wheel[4];
  public Door
    left = new Door(),
    right = new Door(); // 2-door
  public Car() {
    for(int i = 0; i < 4; i++)
      wheel[i] = new Wheel();
  }
  public static void main(String[] args) {
    Car car = new Car();
    car.left.window.rollup();
    car.wheel[0].inflate(72);
  }
} ///:~
```

257

由于在这个例子中**car**的组合也是问题分析的一部分（而不仅仅是底层设计的一部分），所以使成员成为**public**将有助于客户端程序员了解怎样去使用类，而且也降低了类开发者所面临的代码复杂度。但务必要记得这仅仅是一个特例，一般情况下应该使域成为**private**。

在继承的时候，使用某个现有类，并开发一个它的特殊版本。通常，这意味着你在使用一个通用类，并为了某种特殊需要而将其特殊化。略微思考一下就会发现，用一个“交通工具”

对象来构成一部“车子”是毫无意义的，因为“车子”并不包含“交通工具”，它仅是一种交通工具（is-a关系）。“is-a”（是一个）的关系是用继承来表达的，而“has-a”（有一个）的关系则是用组合来表达的。

练习14：(1) 在**Car.java**中给**Engine**添加一个**service()**方法，并在**main()**中调用该方法。

7.6 protected关键字

现在，我们已介绍完了继承，关键字**protected**最终具有了意义。在理想世界中，仅靠关键字**private**就已经足够了。但在实际项目中，经常会想要将某些事物尽可能对这个世界隐藏起来，但仍然允许导出类的成员访问它们。

关键字**protected**就是起这个作用的。它指明“就类用户而言，这是**private**的，但对于任何继承于此类的导出类或其他任何位于同一个包内的类来说，它却是可以访问的。”（**protected**也提供了包内访问权限。）

尽管可以创建**protected**域，但是最好的方式还是将域保持为**private**，你应当一直保留“更改底层实现”的权利。然后通过**protected**方法来控制类的继承者的访问权限。

258

```
//: reusing/Orc.java
// The protected keyword.
import static net.mindview.util.Print.*;

class Villain {
  private String name;
  protected void set(String nm) { name = nm; }
  public Villain(String name) { this.name = name; }
  public String toString() {
    return "I'm a Villain and my name is " + name;
  }
}

public class Orc extends Villain {
  private int orcNumber;
  public Orc(String name, int orcNumber) {
    super(name);
    this.orcNumber = orcNumber;
  }
  public void change(String name, int orcNumber) {
    set(name); // Available because it's protected
    this.orcNumber = orcNumber;
  }
  public String toString() {
    return "Orc " + orcNumber + ": " + super.toString();
  }
  public static void main(String[] args) {
    Orc orc = new Orc("Limburger", 12);
    print(orc);
    orc.change("Bob", 19);
    print(orc);
  }
} /* Output:
Orc 12: I'm a Villain and my name is Limburger
Orc 19: I'm a Villain and my name is Bob
*///:~
```

可以发现，**change()**可以访问**set()**，这是因为它是**protected**的。还应注意**Orc**的**toString()**方法的定义方式，它依据**toString()**的基类版本而定义。

259

练习15：(2) 在包中编写一个类，类应具备一个**protected**方法。在包外部，试着调用该**protected**方法并解释其结果。然后，从你的类中继承产生一个类，并从该导出类的方法内部调

用该**protected**方法。

7.7 向上转型

“为新的类提供方法”并不是继承技术中最重要的方面，其最重要的方面是用来表现新类和基类之间的关系。这种关系可以用“新类是现有类的一种类型”这句话加以概括。

这个描述并非只是一种解释继承的华丽的方式，这直接是由语言所支撑的。例如，假设有一个称为**Instrument**的代表乐器的基类和一个称为**Wind**的导出类。由于继承可以确保基类中所有的方法在导出类中也同样有效，所以能够向基类发送的所有信息同样也可以向导出类发送。如果**Instrument** 类具有一个**play()**方法，那么**Wind**乐器也将同样具备。这意味着我们可以准确地说**Wind**对象也是一种类型的**Instrument**。下面这个例子说明了编译器是怎样支持这一概念的：

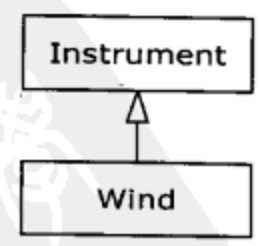
```
//: reusing/Wind.java
// Inheritance & upcasting.

class Instrument {
  public void play() {}
  static void tune(Instrument i) {
    // ...
    i.play();
  }
}

// Wind objects are instruments
// because they have the same interface:
public class Wind extends Instrument {
  public static void main(String[] args) {
    Wind flute = new Wind();
    Instrument.tune(flute); // Upcasting
  }
} ///:~
```

260

在此例中，**tune()**方法可以接受**Instrument** 引用，这实在太有趣了。但在**Wind.main()**中，传递给**tune()**方法的是一个**Wind** 引用。鉴于Java对类型的检查十分严格，接受某种类型的方法同样可以接受另外一种类型就会显得很奇怪，除非你认识到**Wind**对象同样也是一种**Instrument**对象，而且也不存在任何**tune()**方法是可以通过**Instrument**来调用，同时又不存在于**Wind**之中。在**tune()**中，程序代码可以对**Instrument**和它所有的导出类起作用，这种将**Wind**引用转换为**Instrument** 引用的动作，我们称之为*向上转型*。

7.7.1 为什么称为向上转型

该术语的使用有其历史原因，并且是以传统的类继承图的绘制方法为基础的：将根置于页面的顶端，然后逐渐向下。（当然也可以以任何你认为有效的方法进行绘制。）于是，**Wind.java**的继承图就是（如右图所示）：



由导出类转型成基类，在继承图上是向上移动的，因此一般称为*向上转型*。由于向上转型是从一个较专用类型向较通用类型转换，所以总是很安全的。也就是说，导出类是基类的一个超集。它可能比基类含有更多的方法，但它必须至少具备基类中所含有的方法。在向上转型的过程中，类接口中唯一可能发生的事情是丢失方法，而不是获取它们。这就是为什么编译器在“未曾明确表示转型”或“未曾指定特殊标记”的情况下，仍然允许向上转型的原因。

也可以执行与向上转型相反的*向下转型*，但其中含有一个难题，这将在第8章和第14章中进一步解释。

7.7.2 再论组合与继承

在面向对象编程中，生成和使用程序代码最有可能采用的方法就是直接将数据和方法包装进一个类中，并使用该类的对象。也可以运用组合技术使用现有类来开发新的类；而继承技术其实是不太常用的。因此，尽管在教授OOP的过程中我们多次强调继承，但这并不意味着要尽可能使用它。相反，应当慎用这一技术，其使用场合仅限于你确信使用该技术确实有效的情况。到底是该用组合还是用继承，一个最清晰的判断办法就是问一问自己是否需要从新类向基类进行向上转型。如果必须向上转型，则继承是必要的；但如果不需要，则应当好好考虑自己是否需要继承。第8章提出了一个使用向上转型的最具说服力的理由，但只要记得自问一下“我真的需要向上转型吗？”就能较好地在这两种技术中做出决定。

261

练习16：(2) 创建一个名为**Amphibian**的类。由此继承产生一个称为**Frog**的类。在基类中设置适当的方法。在**main()**中，创建一个**Frog**并向上转型至**Amphibian**，然后说明所有方法都可工作。

练习17：(1) 修改练习16，使**Frog**覆盖基类中方法的定义（令新定义使用相同的方法特征签名）。请留心**main()**中都发生了什么。

7.8 final关键字

根据上下文环境，Java的关键字**final**的含义存在着细微的区别，但通常它指的是“这是无法改变的。”不想做改变可能出于两种理由：设计或效率。由于这两个原因相差很远，所以关键字**final**有可能被误用。

以下几节谈论了可能使用到**final**的三种情况：数据、方法和类。

7.8.1 final 数据

许多编程语言都有某种方法，来向编译器告知一块数据是恒定不变的。有时数据的恒定不变是很有用的，比如：

1. 一个永不改变的编译时常量。
2. 一个在运行时被初始化的值，而你不希望它被改变。

对于编译期常量这种情况，编译器可以将该常量值代入任何可能用到它的计算式中，也就是说，可以在编译时执行计算式，这减轻了一些运行时的负担。在Java中，这类常量必须是基本数据类型，并且以关键字**final**表示。在对这个常量进行定义的时候，必须对其进行赋值。

262

一个既是**static**又是**final**的域只占据一段不能改变的存储空间。

当对对象引用而不是基本类型运用**final**时，其含义会有一点令人迷惑。对于基本类型，**final**使数值恒定不变；而用于对象引用，**final**使引用恒定不变。一旦引用被初始化指向一个对象，就无法再把它改为指向另一个对象。然而，对象其自身却是可以被修改的，Java并未提供使任何对象恒定不变的途径（但可以自己编写类以取得使对象恒定不变的效果）。这一限制同样适用数组，它也是对象。

下面的示例示范了**final**域的情况。注意，根据惯例，既是**static**又是**final**的域（即编译期常量）将用大写表示，并使用下划线分隔各个单词：

```
//: reusing/FinalData.java
// The effect of final on fields.
import java.util.*;
import static net.mindview.util.Print.*;

class Value {
  int i; // Package access
```

```
  public Value(int i) { this.i = i; }
}

public class FinalData {
  private static Random rand = new Random(47);
  private String id;
  public FinalData(String id) { this.id = id; }
  // Can be compile-time constants:
  private final int valueOne = 9;
  private static final int VALUE_TWO = 99;
  // Typical public constant:
  public static final int VALUE_THREE = 39;
  // Cannot be compile-time constants:
  private final int i4 = rand.nextInt(20);
  static final int INT_5 = rand.nextInt(20);
  private Value v1 = new Value(11);
  private final Value v2 = new Value(22);
  private static final Value VAL_3 = new Value(33);
  // Arrays:
  private final int[] a = { 1, 2, 3, 4, 5, 6 };
  public String toString() {
    return id + ": " + "i4 = " + i4 + ", INT_5 = " + INT_5;
  }
  public static void main(String[] args) {
    FinalData fd1 = new FinalData("fd1");
    //! fd1.valueOne++; // Error: can't change value
    fd1.v2.i++; // Object isn't constant!
    fd1.v1 = new Value(9); // OK -- not final
    for(int i = 0; i < fd1.a.length; i++)
      fd1.a[i]++; // Object isn't constant!
    //! fd1.v2 = new Value(0); // Error: Can't
    //! fd1.VAL_3 = new Value(1); // change reference
    //! fd1.a = new int[3];
    print(fd1);
    print("Creating new FinalData");
    FinalData fd2 = new FinalData("fd2");
    print(fd1);
    print(fd2);
  }
} /* Output:
fd1: i4 = 15, INT_5 = 18
Creating new FinalData
fd1: i4 = 15, INT_5 = 18
fd2: i4 = 13, INT_5 = 18
*///:~
```

263

由于**valuOne**和**VAL_TWO**都是带有编译时数值的**final**基本类型，所以它们二者均可以用作编译期常量，并且没有重大区别。**VAL_THREE**是一种更加典型的对常量进行定义的方式：定义为**public**，则可以被用于包之外；定义为static，则强调只有一份；定义为**final**，则说明它是一个常量。请注意，带有恒定初始值（即，编译期常量）的**final static**基本类型全用大写字母命名，并且字与字之间用下划线隔开（这就像C常量一样，C常量是这一命名传统的发源地）。

我们不能因为某数据是**final**的就认为在编译时可以知道它的值。在运行时使用随机生成的数值来初始化**i4**和**INT_5**就说明了这一点。示例部分也展示了将**final**数值定义为静态和非静态的区别。此区别只有当数值在运行时内被初始化时才会显现，这是因为编译器对编译时数值一视同仁（并且它们可能因优化而消失）。当运行程序时就会看到这个区别。请注意，在**fd1**和**fd2**中，**i4**的值是唯一的，但**INT_5**的值是不可以通过创建第二个**FinalData**对象而加以改变的。这是因为它是**static**的，在装载时已被初始化，而不是每次创建新对象时都初始化。

264

v1到**VAL_3**这些变量说明了**final**引用的意义。正如在**main()**中所看到的，不能因为**v2**是**final**的，就认为无法改变它的值。由于它是一个引用，**final**意味着无法将**v2**再次指向另一个新的对

象。这对数组具有同样的意义，数组只不过是另一种引用（我还不知道有什么办法能使数组引用本身成为**final**）。看起来，使引用成为**final**没有使基本类型成为**final**的用处大。

练习18：(2) 创建一个含有**static final**域和**final**域的类，说明二者间的区别。

空白final

Java允许生成“空白**final**”，所谓空白**final**是指被声明为**final**但又未给定初值的域。无论什么情况，编译器都确保空白**final**在使用前必须被初始化。但是，空白**final**在关键字**final**的使用上提供了更大的灵活性，为此，一个类中的**final**域就可以做到根据对象而有所不同，却又保持其恒定不变的特性。下面即为一例：

```
//: reusing/BlankFinal.java
// "Blank" final fields.

class Poppet {
  private int i;
  Poppet(int ii) { i = ii; }
}

public class BlankFinal {
  private final int i = 0; // Initialized final
  private final int j; // Blank final
  private final Poppet p; // Blank final reference
  // Blank finals MUST be initialized in the constructor:
  public BlankFinal() {
    j = 1; // Initialize blank final
    p = new Poppet(1); // Initialize blank final reference
  }
  public BlankFinal(int x) {
    j = x; // Initialize blank final
    p = new Poppet(x); // Initialize blank final reference
  }
  public static void main(String[] args) {
    new BlankFinal();
    new BlankFinal(47);
  }
} ///:~
```

265

必须在域的定义处或者每个构造器中用表达式对**final**进行赋值，这正是**final**域在使用前总是被初始化的原因所在。

练习19：(2) 创建一个含有指向某对象的空白**final**引用的类。在所有构造器内部都执行空白**final**的初始化动作。说明Java确保**final**在使用前必须被初始化，且一旦被初始化即无法改变。

final 参数

Java允许在参数列表中以声明的方式将参数指明为**final**。这意味着你无法在方法中更改参数引用所指向的对象：

```
//: reusing/FinalArguments.java
// Using "final" with method arguments.

class Gizmo {
  public void spin() {}
}

public class FinalArguments {
  void with(final Gizmo g) {
    //! g = new Gizmo(); // Illegal -- g is final
  }
  void without(Gizmo g) {
    g = new Gizmo(); // OK -- g not final
    g.spin();
  }
```

```
  // void f(final int i) { i++; } // Can't change
  // You can only read from a final primitive:
  int g(final int i) { return i + 1; }
  public static void main(String[] args) {
    FinalArguments bf = new FinalArguments();
    bf.without(null);
    bf.with(null);
  }
} ///:~
```

266

方法f()和g()展示了当基本类型的参数被指明为**final**时所出现的结果：你可以读参数，但却无法修改参数。这一特性主要用来向匿名内部类传递数据，我们将在第10章中学习它。

7.8.2 final 方法

使用**final**方法的原因有两个。第一个原因是把方法锁定，以防任何继承类修改它的含义。这是出于设计的考虑：想要确保在继承中使方法行为保持不变，并且不会被覆盖。

过去建议使用**final**方法的第二个原因是效率。在Java的早期实现中，如果将一个方法指明为**final**，就是同意编译器将针对该方法的所有调用都转为内嵌调用。当编译器发现一个**final**方法调用命令时，它会根据自己的谨慎判断，跳过插入程序代码这种正常方式而执行方法调用机制（将参数压入栈，跳至方法代码处并执行，然后跳回并清理栈中的参数，处理返回值），并且以方法体中的实际代码的副本来替代方法调用。这将消除方法调用的开销。当然，如果一个方法很大，你的程序代码就会膨胀，因而可能看不到内嵌带来的任何性能提高，因为，所带来的性能提高会因为花费于方法内的时间量而被缩减。

在最近的Java版本中，虚拟机（特别是hotspot技术）可以探测到这些情况，并优化去掉这些效率反而降低的额外的内嵌调用，因此不再需要使用**final**方法来进行优化了。事实上，这种做法正在逐渐地受到劝阻。在使用Java SE5/6时，应该让编译器和JVM去处理效率问题，只有在想要明确禁止覆盖时，才将方法设置为**final**的[⊖]。

267

final和private关键字

类中所有的**private**方法都隐式地指定为是**final**的。由于无法取用**private**方法，所以也就无法覆盖它。可以对**private**方法添加**final** 修饰词，但这并不能给该方法增加任何额外的意义。

这一问题会造成混淆。因为，如果你试图覆盖一个**private**方法（隐含是**final**的），似乎是奏效的，而且编译器也不会给出错误信息：

```
//: reusing/FinalOverridingIllusion.java
// It only looks like you can override
// a private or private final method.
import static net.mindview.util.Print.*;

class WithFinals {
  // Identical to "private" alone:
  private final void f() { print("WithFinals.f()"); }
  // Also automatically "final":
  private void g() { print("WithFinals.g()"); }
}

class OverridingPrivate extends WithFinals {
  private final void f() {
    print("OverridingPrivate.f()");
  }
```

⊖ 不要陷入对仓促优化的强烈渴望之中。如果你的系统得以运行，而其速度很慢，使用**final**关键字来修复该问题是难以奏效的。在http://MindView.net/Books/BetterJava可以找到有关性能测试的信息，它们有助于提高你的程序的运行速度。

```
    private void g() {
      print("OverridingPrivate.g()");
    }
  }

  class OverridingPrivate2 extends OverridingPrivate {
    public final void f() {
      print("OverridingPrivate2.f()");
    }
    public void g() {
      print("OverridingPrivate2.g()");
    }
  }

  public class FinalOverridingIllusion {
    public static void main(String[] args) {
      OverridingPrivate2 op2 = new OverridingPrivate2();
      op2.f();
      op2.g();
      // You can upcast:
      OverridingPrivate op = op2;
      // But you can't call the methods:
      //! op.f();
      //! op.g();
      // Same here:
      WithFinals wf = op2;
      //! wf.f();
      //! wf.g();
    }
  } /* Output:
  OverridingPrivate2.f()
  OverridingPrivate2.g()
  *///:~
```

268

"覆盖"只有在某方法是基类的接口的一部分时才会出现。即，必须能将一个对象向上转型为它的基本类型并调用相同的方法（这一点在下一章阐明）。如果某方法为**private**，它就不是基类的接口的一部分。它仅是一些隐藏于类中的程序代码，只不过是具有相同的名称而已。但如果在导出类中以相同的名称生成一个**public**、**protected**或包访问权限方法的话，该方法就不会产生在基类中出现的"仅具有相同名称"的情况。此时你并没有覆盖该方法，仅是生成了一个新的方法。由于**private**方法无法触及而且能有效隐藏，所以除了把它看成是因为它所归属的类的组织结构的原因而存在外，其他任何事物都不需要考虑到它。

练习20：(1) 展示**@Override**注解可以解决本节中的问题。

269

练习21：(1) 创建一个带**final**方法的类。由此继承产生一个类并尝试覆盖该方法。

7.8.3 final 类

当将某个类的整体定义为**final**时（通过将关键字**final**置于它的定义之前），就表明了你不打算继承该类，而且也不允许别人这样做。换句话说，出于某种考虑，你对该类的设计永不需要做任何变动，或者出于安全的考虑，你不希望它有子类。

```
//: reusing/Jurassic.java
// Making an entire class final.

class SmallBrain {}

final class Dinosaur {
  int i = 7;
  int j = 1;
  SmallBrain x = new SmallBrain();
  void f() {}
}
```

```
//! class Further extends Dinosaur {}
// error: Cannot extend final class 'Dinosaur'

public class Jurassic {
  public static void main(String[] args) {
    Dinosaur n = new Dinosaur();
    n.f();
    n.i = 40;
    n.j++;
  }
} ///:~
```

请注意，**final**类的域可以根据个人的意愿选择为是或不是**final**。不论类是否被定义为**final**，相同的规则都适用于定义为**final**的域。然而，由于**final**类禁止继承，所以**final**类中所有的方法都隐式指定为是**final**的，因为无法覆盖它们。在**final**类中可以给方法添加**final**修饰词，但这不会增添任何意义。

练习22：(1) 创建一个**final**类并试着继承它。 270

7.8.4 有关final的忠告

在设计类时，将方法指明是**final**的，应该说是明智的。你可能会觉得，没人会想要覆盖你的方法。有时这是对的。

但请留意你所作的假设。要预见类是如何被复用的一般是很困难的，特别是对于一个通用类而言更是如此。如果将一个方法指定为**final**，可能会妨碍其他程序员在项目中通过继承来复用你的类，而这只是因为你没有想到它会以那种方式被运用。

Java标准程序库就是一个很好的例子。特别是Java 1.0/1.1中**Vector**类被广泛地运用，而且从效率考虑（这近乎是一个幻想），如果所有的方法均未被指定为**final**的话，它可能会更加有用。很容易想像到，人们可能会想要继承并覆盖如此基础而有用的类，但是设计者却认为这样做不太合适。这里有两个令人意外的原因。第一，**Stack**继承自**Vector**，就是说**Stack**是个**Vector**，这从逻辑的观点看是不正确的。尽管如此，Java的设计者们自己仍旧继承了**Vector**。在以这种方式创建**Stack**时，他们应该意识到**final**方法显得过于严苛了。

第二，**Vector**的许多最重要的方法—如**addElement()**和**elementAt()**是同步的。正如在第21章中将要看到的那样，这将导致很大的执行开销，可能会抹煞**final**所带来的好处。这种情况增强了人们关于程序员无法正确猜测优化应当发生于何处的观点。如此蹩脚的设计，却要置于我们每个人都得使用的标准程序库中，这是很糟糕的（幸运的是，现代Java的容器库用**ArrayList**替代了**Vector**。**ArrayList**的行为要合理得多。遗憾的是仍然存在用旧容器库编写新程序代码的情况）。

留心一下**Hashtable**，这个例子同样有趣，它也是一个重要的Java1.0/1.1标准库类，而且不含任何**final**方法。如本书其他地方所提到的，某些类明显是由一些互不相关的人设计的（读者会发现，名为**Hashtable**的方法相对于**Vector**中的方法要简洁得多，这又是一个证据）。对于类库的使用者来说，这又是一个本不该如此轻率的事物。这种不规则的情况只能使用户付出更多的努力。这是对粗糙的设计和代码的又一讽刺（请注意，现代Java的容器库用**HashMap**替代了**Hashtable**）。 271

7.9 初始化及类的加载

在许多传统语言中，程序是作为启动过程的一部分立刻被加载的。然后是初始化，紧接着程序开始运行。这些语言的初始化过程必须小心控制，以确保定义为**static**的东西，其初始化顺

序不会造成麻烦。例如C++中，如果某个**static**期望另一个**static**在被初始化之前就能有效地使用它，那么就会出现问题。

Java就不会出现这个问题，因为它采用了一种不同的加载方式。加载是众多变得更加容易的动作之一，因为Java中的所有事物都是对象。请记住，每个类的编译代码都存在于它自己的独立的文件中。该文件只在需要使用程序代码时才会被加载。一般来说，可以说："类的代码在初次使用时才加载。"这通常是指加载发生于创建类的第一个对象之时，但是当访问**static**域或**static**方法时，也会发生加载[⊖]。

初次使用之处也是**static**初始化发生之处。所有的**static**对象和**static**代码段都会在加载时依程序中的顺序（即，定义类时的书写顺序）而依次初始化。当然，定义为**static**的东西只会被初始化一次。

7.9.1 继承与初始化

了解包括继承在内的初始化全过程，以对所发生的一切有个全局性的把握，是很有益的。请看下例：

272

```
//: reusing/Beetle.java
// The full process of initialization.
import static net.mindview.util.Print.*;
class Insect {
  private int i = 9;
  protected int j;
  Insect() {
    print("i = " + i + ", j = " + j);
    j = 39;
  }
  private static int x1 =
    printInit("static Insect.x1 initialized");
  static int printInit(String s) {
    print(s);
    return 47;
  }
}

public class Beetle extends Insect {
  private int k = printInit("Beetle.k initialized");
  public Beetle() {
    print("k = " + k);
    print("j = " + j);
  }
  private static int x2 =
    printInit("static Beetle.x2 initialized");
  public static void main(String[] args) {
    print("Beetle constructor");
    Beetle b = new Beetle();
  }
} /* Output:
static Insect.x1 initialized
static Beetle.x2 initialized
Beetle constructor
i = 9, j = 0
Beetle.k initialized
k = 47
j = 39
*///:~
```

在**Beetle**上运行Java时，所发生的第一件事情就是试图访问**Beetle.main()**（一个**static**方法），

⊖ 构造器也是**static**方法，尽管**static**关键字并没有显式地写出来。因此更准确地讲，类是在其任何**static**成员被访问时加载的。

于是加载器开始启动并找出**Beetle**类的编译代码（在名为**Beetle.class**的文件之中）。在对它进行加载的过程中，编译器注意到它有一个基类（这是由关键字extends得知的），于是它继续进行加载。不管你是否打算产生一个该基类的对象，这都要发生（请尝试将对象创建代码注释掉，以证明这一点）。

273

如果该基类还有其自身的基类，那么第二个基类就会被加载，如此类推。接下来，根基类中的**static**初始化（在此例中为**Insect**）即会被执行，然后是下一个导出类，以此类推。这种方式很重要，因为导出类的**static**初始化可能会依赖于基类成员能否被正确初始化。

至此为止，必要的类都已加载完毕，对象就可以被创建了。首先，对象中所有的基本类型都会被设为默认值，对象引用被设为**null**——这是通过将对象内存设为二进制零值而一举生成的。然后，基类的构造器会被调用。在本例中，它是被自动调用的。但也可以用**super**来指定对基类构造器的调用（正如在**Beetle()**构造器中的第一步操作）。基类构造器和导出类的构造器一样，以相同的顺序来经历相同的过程。在基类构造器完成之后，实例变量按其次序被初始化。最后，构造器的其余部分被执行。

练习23：(2) 请证明加载类的动作仅发生一次。证明该类的第一个实体的创建或者对**static**成员的访问都有可能引起加载。

练习24：(2) 在**Beetle.java**中，从**Beetle**类继承产生一个具体类型的“甲壳虫”。其形式与现有类相同，跟踪并解释其输出结果。

7.10 总结

继承和组合都能从现有类型生成新类型。组合一般是将现有类型作为新类型底层实现的一部分来加以复用，而继承复用的是接口。

在使用继承时，由于导出类具有基类接口，因此它可以向上转型至基类，这对多态来讲至关重要，就像我们将在下一章中将要看到的那样。

尽管面向对象编程对继承极力强调，但在开始一个设计时，一般应优先选择使用组合（或者可能是代理），只在确实必要时才使用继承。因为组合更具灵活性。此外，通过对成员类型使用继承技术的添加技巧，可以在运行时改变那些成员对象的类型和行为。因此，可以在运行时改变组合而成的对象的行为。

274

在设计一个系统时，目标应该是找到或创建某些类，其中每个类都有具体的用途，而且既不会太大（包含太多的功能而难以复用），也不会太小（不添加其他功能就无法使用）。如果你的设计变得过于复杂，通过将现有类拆分为更小的部分而添加更多的对象，通常会有所帮助。

当你开始设计一个系统时，应该认识到程序开发是一种增量过程，犹如人类的学习一样，这一点很重要。程序开发依赖于实验，你可以尽己所能去分析，但当你开始执行一个项目时，你仍然无法知道所有的答案。如果将项目视作是一种有机的、进化着的生命体而去培养，而不是打算像盖摩天大楼一样快速见效，就会获得更多的成功和更迅速的回馈。继承与组合正是在面向对象程序设计中使得你可以执行这种实验的最基本的两个工具。

275 ~ 276

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。